

RELATÓRIO DE TOMADAS DE DECISÃO

Projeto: Plataforma de BI Analítico Estratégico com IA e Terminal SGBD Interativo

1. Introdução

Este documento apresenta as principais decisões tomadas durante o desenvolvimento da Plataforma de BI Analítico Estratégico com IA e Terminal SGBD Interativo. O objetivo do relatório é justificar tecnicamente as escolhas realizadas pela equipe durante o processo de desenvolvimento, demonstrando os critérios utilizados para selecionar tecnologias, arquiteturas e metodologias capazes de atender aos requisitos funcionais e acadêmicos do projeto.

O sistema foi desenvolvido para integrar conceitos de desenvolvimento Full Stack, Business Intelligence (BI), Banco de Dados e Inteligência Artificial em uma única aplicação web interativa.

2. Metodologia de Tomada de Decisão

As decisões foram tomadas considerando os seguintes critérios:

- Facilidade de desenvolvimento.
- Facilidade de aprendizado para fins acadêmicos.
- Desempenho adequado ao escopo do projeto.
- Baixo custo de implementação.
- Manutenibilidade do código.
- Escalabilidade futura.
- Integração entre módulos de software.

Cada tecnologia escolhida foi comparada com alternativas disponíveis no mercado, sendo selecionada a que melhor atendia às necessidades do projeto.

3. Decisões Técnicas

3.1 Escolha da Arquitetura Full Stack

Problema

Era necessário desenvolver uma aplicação capaz de realizar gerenciamento de dados, processamento analítico, visualização de informações e execução de algoritmos de inteligência artificial.

Alternativas Avaliadas

- Aplicação exclusivamente Frontend.
- Aplicação Monolítica.
- Arquitetura Full Stack com Frontend e Backend integrados.

Decisão Tomada

Foi adotada uma arquitetura Full Stack composta por React no frontend e Nest no backend.

Justificativa

Essa abordagem permite separar responsabilidades entre interface e processamento de dados, facilitando manutenção, evolução e organização do sistema.

Benefícios

- Melhor organização do código.
 - Separação entre interface e regras de negócio.
 - Facilidade de expansão futura.
-

3.2 Escolha do React para o Frontend

Problema

Era necessário construir uma interface moderna, responsiva e altamente interativa.

Alternativas Avaliadas

- React
- Angular
- Vue.js

Decisão Tomada

Utilização do React com TypeScript.

Justificativa

React oferece grande flexibilidade para construção de componentes reutilizáveis, ampla comunidade e excelente integração com bibliotecas gráficas utilizadas pelo projeto.

Benefícios

- Componentização.
- Reutilização de código.
- Facilidade de manutenção.
- Alto desempenho na renderização da interface.

Desvantagens

- Necessidade de gerenciamento de estados.
 - Curva de aprendizado inicial.
-

3.3 Utilização do TypeScript

Problema

O crescimento do sistema aumentaria a complexidade do código e a possibilidade de erros.

Alternativas Avaliadas

- JavaScript puro.
- TypeScript.

Decisão Tomada

Utilização do TypeScript em todo o projeto.

Justificativa

A tipagem estática reduz erros durante o desenvolvimento e facilita a manutenção do código.

Benefícios

- Maior confiabilidade.
 - Melhor documentação implícita.
 - Melhor suporte das IDEs.
-

3.4 Escolha do NestJS para o Backend

Problema

Era necessário desenvolver uma camada de backend responsável pelo processamento das regras de negócio, gerenciamento das APIs e integração com os demais componentes do sistema.

Alternativas Avaliadas

- Express
- NestJS
- Fastify

Decisão Tomada

Utilização do NestJS como framework principal do backend.

Justificativa

A escolha do NestJS foi motivada principalmente por fazer parte da stack tecnológica obrigatória definida para o desenvolvimento do projeto. Além disso, o framework oferece uma arquitetura baseada em módulos, controladores e serviços, favorecendo a organização do código e a aplicação de boas práticas de desenvolvimento.

Por ser construído sobre o Node.js e possuir suporte nativo ao TypeScript, o NestJS também proporciona maior padronização e manutenibilidade em projetos de médio e grande porte.

Benefícios

- Atendimento aos requisitos tecnológicos do projeto.
- Estrutura organizada baseada em módulos.
- Integração nativa com TypeScript.
- Facilidade de manutenção e escalabilidade.
- Aplicação de padrões de arquitetura amplamente utilizados no mercado.

Desvantagens

- Curva de aprendizado maior em comparação com frameworks mais simples.
 - Maior quantidade de configurações iniciais.
-

3.5 Persistência em Arquivo JSON

Problema

O sistema precisava armazenar clientes, produtos e pedidos.

Alternativas Avaliadas

- MySQL
- PostgreSQL
- SQLite
- Arquivo JSON

Decisão Tomada

Persistência em arquivo JSON.

Justificativa

A escolha permitiu simplificar a infraestrutura do projeto, eliminando a necessidade de instalação e configuração de um banco de dados relacional.

Benefícios

- Facilidade de configuração.
- Portabilidade.
- Simplicidade para demonstrações acadêmicas.

Limitações

- Não adequado para ambientes de produção com grande volume de dados.
 - Baixa escalabilidade.
-

3.6 Implementação do Dashboard de Business Intelligence

Problema

Era necessário transformar dados operacionais em informações estratégicas para apoio à tomada de decisão.

Alternativas Avaliadas

- Relatórios textuais.
- Dashboards gráficos.

Decisão Tomada

Criação de dashboards utilizando a biblioteca Recharts.

Justificativa

Visualizações gráficas permitem identificar tendências e padrões com maior facilidade.

Benefícios

- Melhor experiência do usuário.
 - Facilidade de interpretação dos dados.
 - Suporte a indicadores de desempenho (KPIs).
-

3.7 Implementação do Módulo de Inteligência Artificial

Problema

O sistema precisava fornecer apoio à decisão através da análise de comportamento dos clientes.

Alternativas Avaliadas

- Árvore de Decisão.
- Regressão Logística.
- Random Forest.

Decisão Tomada

Implementação do algoritmo Random Forest em TypeScript.

Justificativa

O Random Forest apresenta maior robustez em comparação a uma única árvore de decisão, reduzindo problemas de overfitting e aumentando a estabilidade das previsões.

Variáveis Utilizadas

- Total gasto pelo cliente.

- Frequência de compras.
- Recência da última compra.
- Ticket médio.
- Taxa de cancelamento.
- Diversidade de produtos adquiridos.

Benefícios

- Melhor capacidade de generalização.
 - Interpretação relativamente simples.
 - Simulação de cenários de churn.
-

3.8 Tratamento e Qualidade dos Dados

Problema

Dados inconsistentes podem comprometer os resultados das análises e previsões.

Decisão Tomada

Implementação de um pipeline de preparação de dados contendo:

- Remoção de registros duplicados.
- Tratamento de outliers utilizando IQR.
- Normalização dos atributos.
- Engenharia de features.

Justificativa

Garantir maior qualidade dos dados utilizados pelo modelo analítico.

Benefícios

- Resultados mais confiáveis.
 - Redução de distorções estatísticas.
 - Melhor desempenho do algoritmo.
-

3.9 Desenvolvimento do Console SQL Educacional

Problema

O projeto precisava demonstrar conceitos de banco de dados relacionais sem depender de um SGBD externo.

Decisão Tomada

Desenvolvimento de um interpretador SQL próprio em TypeScript.

Funcionalidades Implementadas

- SELECT
- WHERE
- ORDER BY
- JOIN
- GROUP BY
- Alias de colunas

Justificativa

A implementação permite demonstrar conceitos fundamentais de bancos de dados de forma prática e integrada ao sistema.

Benefícios

- Valor educacional elevado.
 - Demonstração de conhecimento em processamento de consultas.
 - Integração completa com os dados do sistema.
-

4. Impacto das Decisões no Projeto

As decisões tomadas contribuíram para a construção de uma plataforma integrada capaz de unir conceitos de desenvolvimento Full Stack, Business Intelligence, Banco de Dados e Inteligência Artificial.

A utilização de React, TypeScript e NestJS proporcionou uma arquitetura organizada, modular e de fácil manutenção, favorecendo a separação de responsabilidades entre os componentes do sistema. A adoção do armazenamento em JSON simplificou a implantação do sistema. O uso de dashboards analíticos e do algoritmo Random Forest agregou funcionalidades avançadas de apoio à decisão, enquanto o interpretador SQL reforçou o caráter acadêmico e educacional da solução.

5. Conclusão

O conjunto de decisões adotadas durante o desenvolvimento permitiu alcançar os objetivos propostos pelo projeto, oferecendo uma solução funcional, moderna e alinhada aos conceitos estudados durante a formação acadêmica.

Além de atender aos requisitos técnicos, o sistema demonstra a integração entre áreas fundamentais da computação, como desenvolvimento web, bancos de dados, análise de dados e inteligência artificial, consolidando conhecimentos práticos adquiridos ao longo do curso.